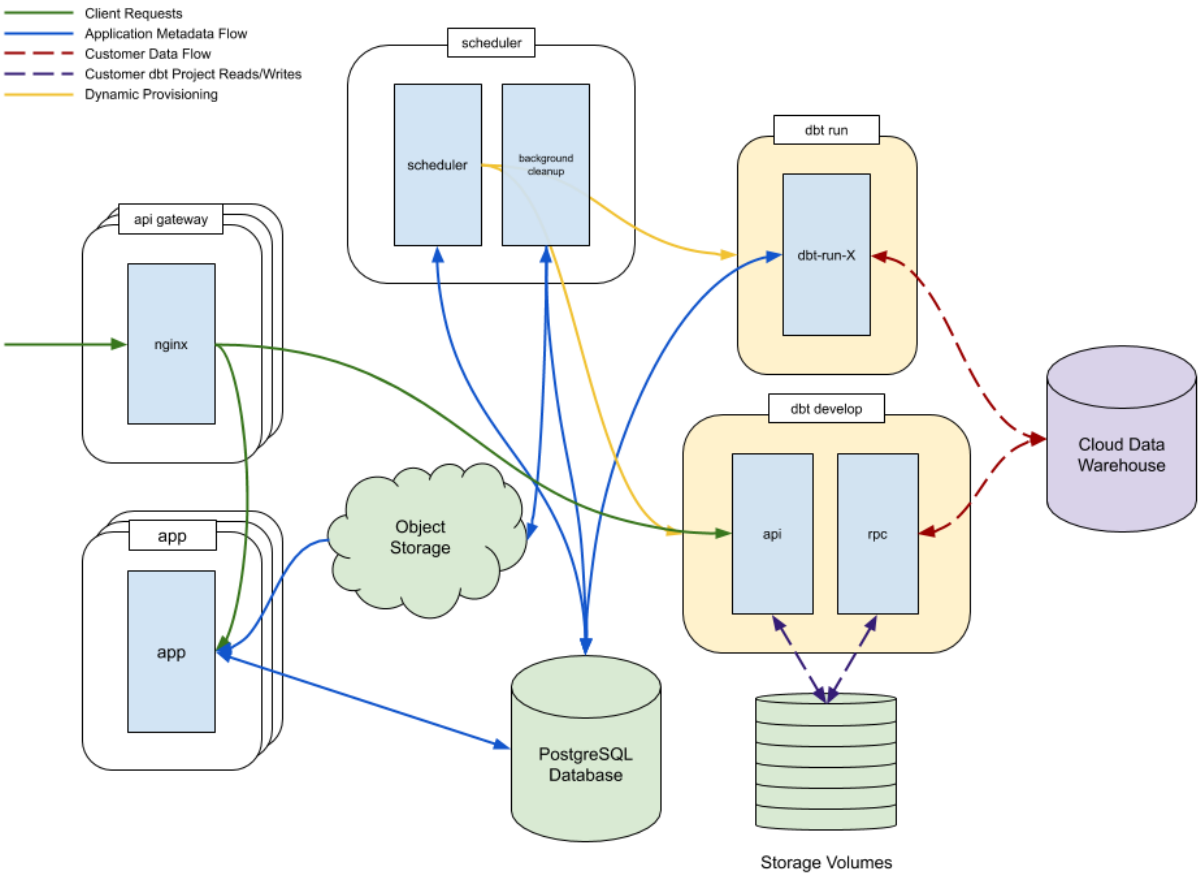
dbt Cloud Advanced Architecture Guide

Application Data Flows

The dbt Cloud application is comprised of a set of static components as well as a set of dynamic components. The static components are constantly running to serve highly available dbt Cloud functionality, for example, the dbt Cloud web application. The dynamic components are created ad-hoc to fill background jobs or a user request to use the IDE. These components are described below.



Static Application Components

- **API gateway:** The API gateway is the entry point for all client requests to dbt Cloud. The API gateway serves static content and contains logic for routing requests within the dbt Cloud application.
- **App:** The app is the dbt Cloud application server. It consists of a Django application that serves dbt Cloud REST API requests.

- **Scheduler:** The scheduler is a continuously running process that orchestrates background jobs in dbt Cloud. It consists of two components: the scheduler container, which provisions dynamic resources just-in-time, and the background cleanup container, which performs maintenance tasks on the dbt Cloud database, including flushing logs from dbt runs out into the object store.

Dynamic Application Components

- **dbt run:** A "run" in dbt Cloud represents a series of background invocations of dbt that are triggered either on a cron scheduler, manually by a user, or via dbt Cloud's API.
- **dbt develop:** This server is capable of serving dbt IDE requests for a single user. dbt Cloud will create one for each user actively using the dbt IDE.

Application Critical Components

In addition to the application components, there are a few critical dependencies of the application components that are required in order for the dbt Cloud application to function.

- **PostgreSQL database:** dbt Cloud uses a PostgreSQL database as its backend. This can be a cloud-hosted database, for example, AWS RDS, Azure Database, Google Cloud SQL (recommended for production deployments); or, it can be embedded into the dbt Cloud Kubernetes appliance (not recommended for production deployments).
- **Object Storage:** dbt Cloud requires an S3-compatible Object Storage system for persisting run logs and artifacts.
- **Storage Volumes:** dbt Cloud requires a Kubernetes storage provider capable of creating dynamic persistent volumes that can be mounted to multiple containers in R/W mode.

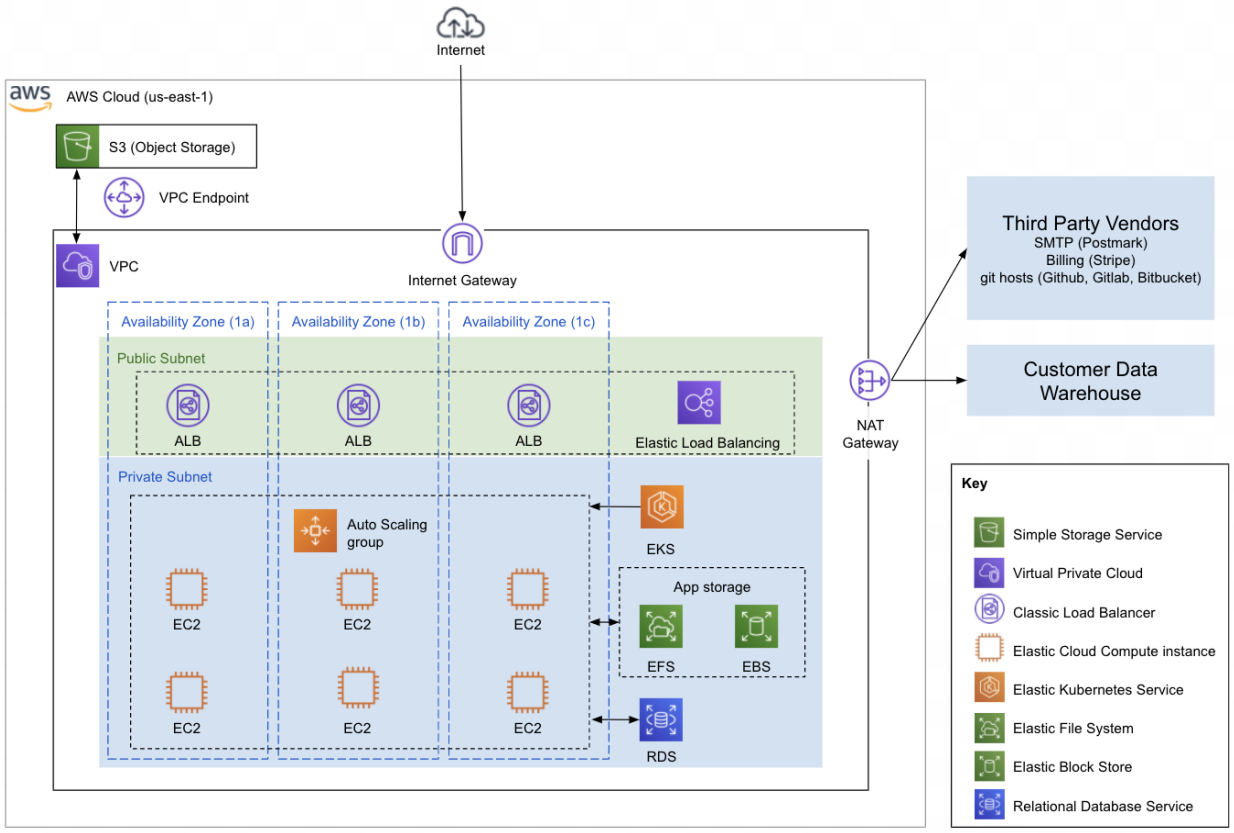
Data Warehouse Interaction

dbt Cloud's primary role is as a data processor, not a data store. The dbt Cloud application lets users deploy SQL to the warehouse for transformation. However, it is possible for users to dispatch SQL that returns customer data into the dbt Cloud application. This data never persists and will only exist in memory on the instance in question. In order to properly lock down customer data, it is critical that proper data warehouse permissions are applied to prevent improper access or storage of sensitive data.

Deployment Architecture

The following two sections describe the network architectures for dbt Cloud deployments. Hosted deployments leverage AWS infrastructure.
Hosted Network Architecture

The following diagram shows the network architecture for the hosted single and multi-tenant deployment types. While many specifications differ between the single and multi-tenant offerings, the basic types of components illustrated below are mostly the same. The following is more information on each component and how they might differ between the two deployment models.



- **VPC:** In both hosted deployments, the dbt Cloud application infrastructure lives in an AWS VPC managed by dbt Labs. One of the key differences between production and single-tenant deployment is that single-tenant deployment provides a dedicated VPC for a single customer.
- **EKS:** Hosted environments leverage AWS Elastic Kubernetes Service to manage dbt Cloud application resources. EKS provides a high degree of reliability and scalability for the dbt Cloud application.
- **CLB:** One or more AWS Classic Load Balancers living in a public subnet are leveraged in the hosted deployment environments to distribute incoming traffic across multiple EC2 instances in the EKS cluster.
- **EC2:** The hosted dbt Cloud deployments leverage a cluster of AWS EC2 worker nodes to run the dbt Cloud application.
- **EBS:** In order to store application data, dbt Cloud leverages AWS Elastic Block Store mounted to the EC2 instances described above.
- **EFS:** An AWS Elastic File System is provisioned for hosted deployments to store and manage local files from the dbt Cloud IDE.

- **S3:** AWS Simple Storage Service (S3) stores dbt Cloud application logs and artifacts (such as those generated from dbt job runs).
- **RDS:** The hosted dbt Cloud application leverages AWS Postgres RDS to store application information such as accounts, users, environments, etc. Note that, as explained in the Data Warehouse Interaction section above, no data from an associated warehouse is ever stored in this database.